CHAPTER 02

Architectural design patterns and best practices

For Business and Technical Analysts in international commercial banking

Antonio Piemontese

October 2026



CHAPTER 02

Learning objectives

01 **Recognise recurring design problems**

Explain the purpose of a pattern before discussing its implementation.

02 **Compare architectural choices**

Relate boundaries, deployment and interaction choices to business needs.

03 **Explain the consequences of a pattern**

Identify added complexity, failure behaviour and responsibility.

04 **Justify a combination with evidence**

Select patterns for the banking case and define useful validation tests.

Patterns address different decisions

Patterns capture reusable responses to recurring design problems.

Several patterns can coexist in the same solution.

Structure

How are responsibilities organised?

Boundaries

Which models and dependencies stay separate?

Interactions

How does work move between parts?

Failure and change

How does the solution behave under stress?

The anatomy of a pattern

Element	Question to ask
Context and problem	What recurring situation needs a design response?
Forces	Which requirements, constraints and tensions matter?
Structure	Which responsibilities and relationships change?
Consequences and evidence	What improves, what becomes harder, and what must be tested?

A pattern is useful only when its context and consequences fit the problem.

HYPOTHETICAL BANKING CASE

Our case: reliable application handoff

Business need

Existing retail customers in one pilot country need a durable receipt while credit assessment continues separately.

Working assumptions

One team delivers the new pilot.

Credit Decision Service owns approval.

Existing bank interfaces remain in place.

API receipt target p95 ≤ 2 s at 20 valid submissions/s.

Principles for clear boundaries

Good boundaries keep related behaviour together and make dependencies explicit.

One module owns application state. Other modules use its interface.

High cohesion

Keep behaviour that changes together close.

Loose coupling

Limit assumptions about another part’s internals.

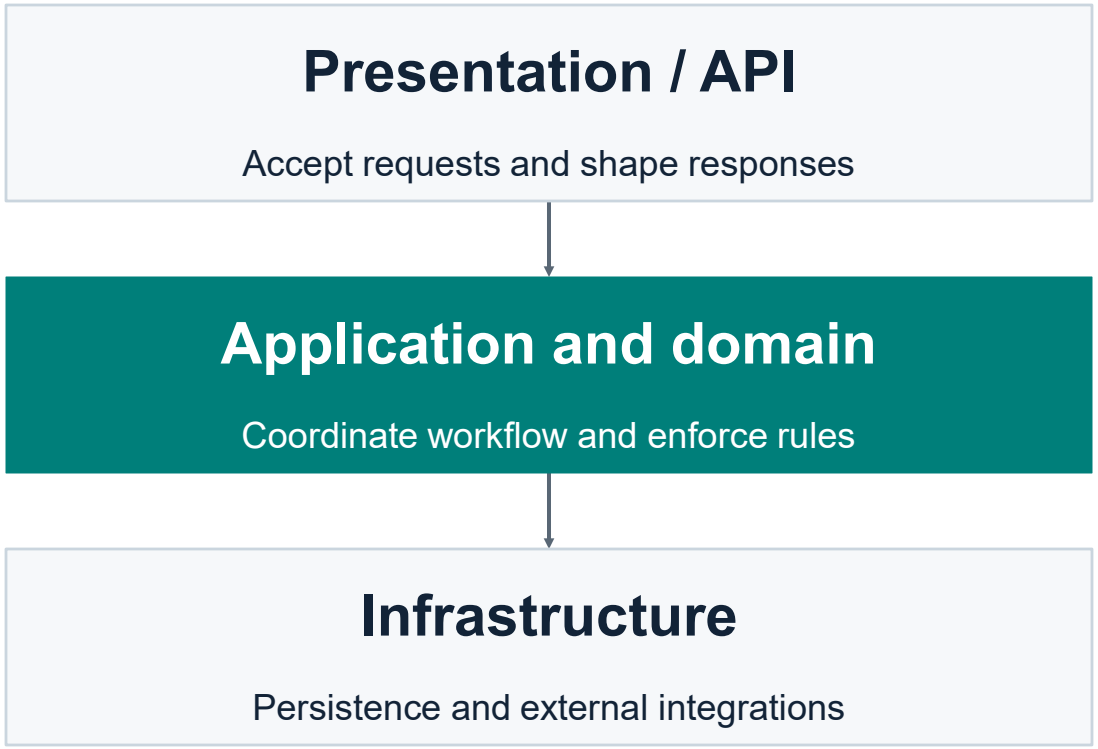
Encapsulation

Keep rules and owned data behind a contract.

Simplicity

Add complexity when a requirement justifies it.

Layered architecture



Useful when

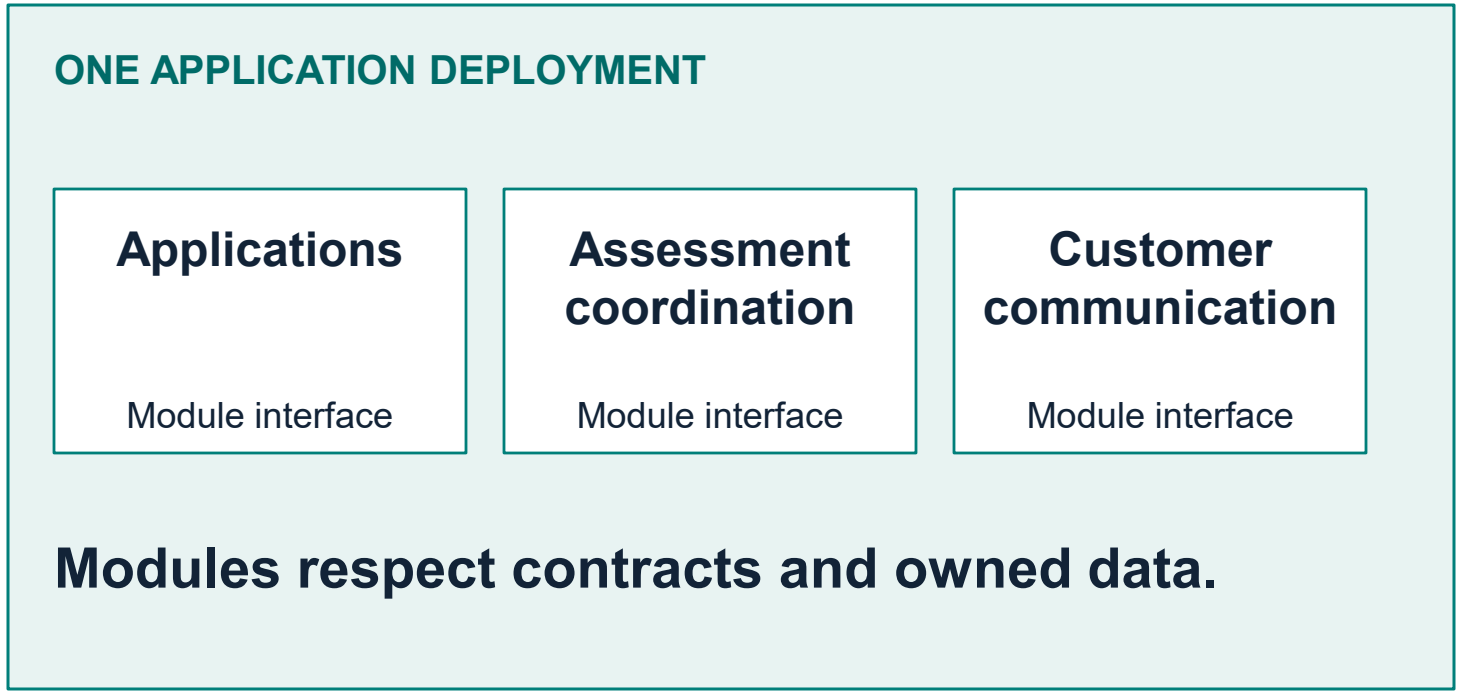
The team needs a clear separation of technical responsibilities.

Trade-off

Changes can spread when layers expose their internals or become catch-all services.

Arrows show calls between logical layers. A layer does not require a separate server.

A modular monolith



Useful when

One team can release the pilot together and clear internal boundaries are enough.

Trade-off

A shared release and runtime still couple deployment and some failure modes.

A shared deployment can still have explicit interfaces, data ownership and multiple running instances.

Ports and adapters



Benefit: test application rules through the same contracts.

Arrows show code dependencies toward application-owned ports. Runtime requests may travel outward.

Bounded contexts and ownership

Application context

Application means submission and progress.

Owns received input and processing status.

Shares identifiers and agreed information.

Credit assessment context

Application means a case to evaluate.

Owns assessment rules and the decision.

Maps incoming data into its own model.

Model boundaries guide ownership. Deployment boundaries remain a separate choice.

Microservices and autonomy

Microservices use independently deployable services with explicit interfaces and ownership.

An existing shared credit service does not require every new module to become a service.

Deployment autonomy

Compatible changes need no joint release.

Data ownership

Other services respect its data boundary.

Remote interaction

Network delay and partial failure become visible.

Operating effort

Each service needs delivery and support capability.

Choosing a deployment structure

Criterion	Modular monolith	Microservices
Release boundary	One application release	Independent service releases
Transactions	Local changes are often simpler	Cross-service changes need coordination
Change isolation	Module interfaces protect boundaries	Contracts protect service boundaries
Operating effort	Fewer separately operated parts	More delivery and runtime responsibilities

For this pilot, the need for independent deployment must justify the added work.

Request–reply and asynchronous messaging

Synchronous request–reply

The caller waits for a response.

Runtime availability affects the call.

Timeouts can leave the outcome uncertain.

Asynchronous messaging

The sender hands work to a channel.

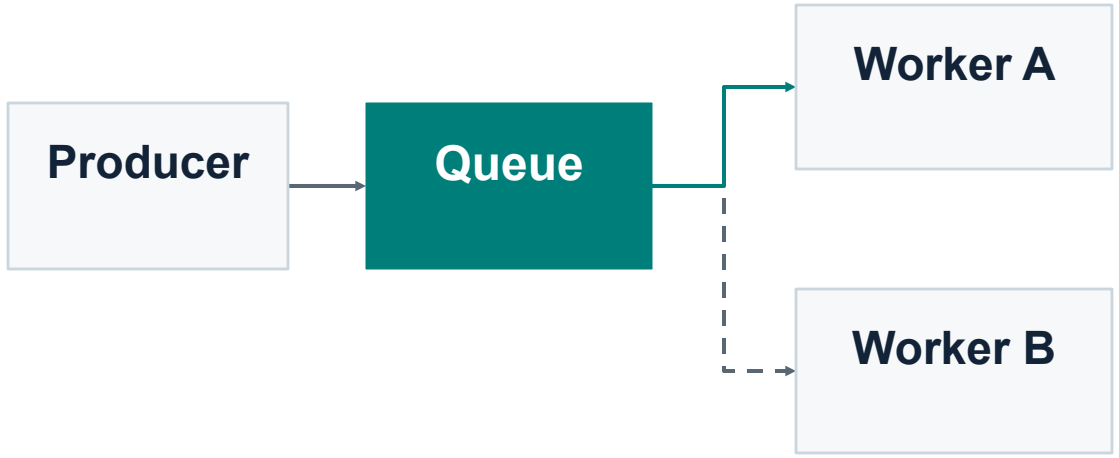
Processing can continue independently.

Delivery, ordering and retries need a contract.

Business timing and failure requirements determine the appropriate interaction.

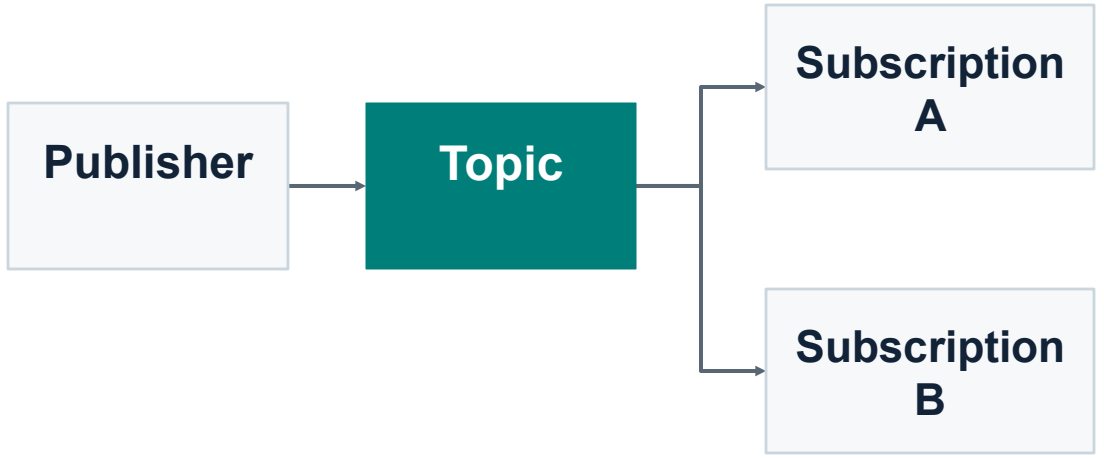
Publish–subscribe and work queues

Competing consumers



One worker per delivery.
The dashed path is an alternative.

Publish–subscribe



Each matching subscription receives its own copy.

Both models can redeliver messages. Retention, ordering and delivery guarantees depend on the channel contract.

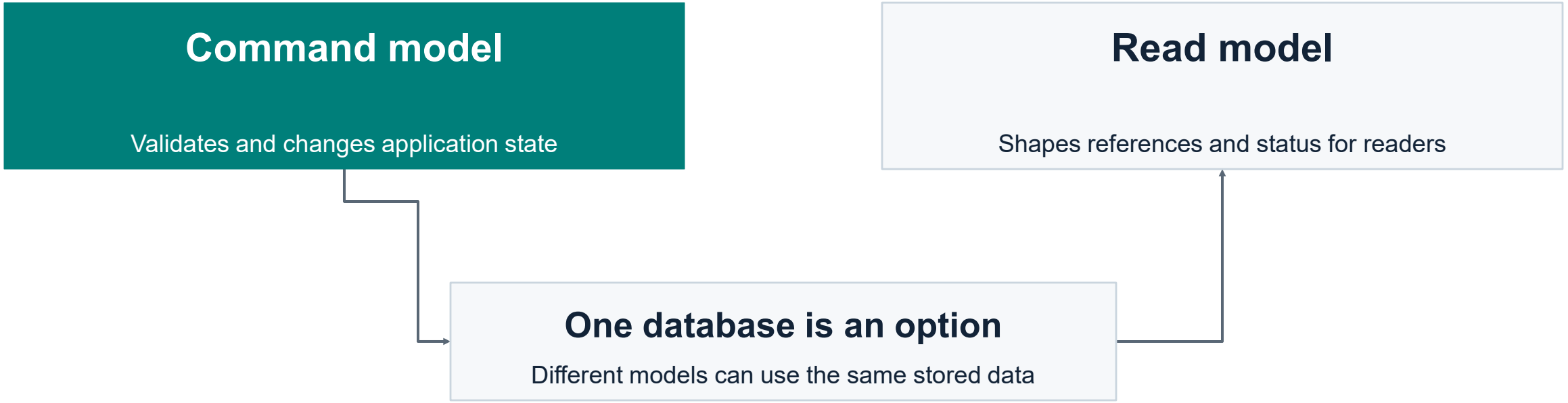
CQRS: separating writes and reads

Write side

Submit an application

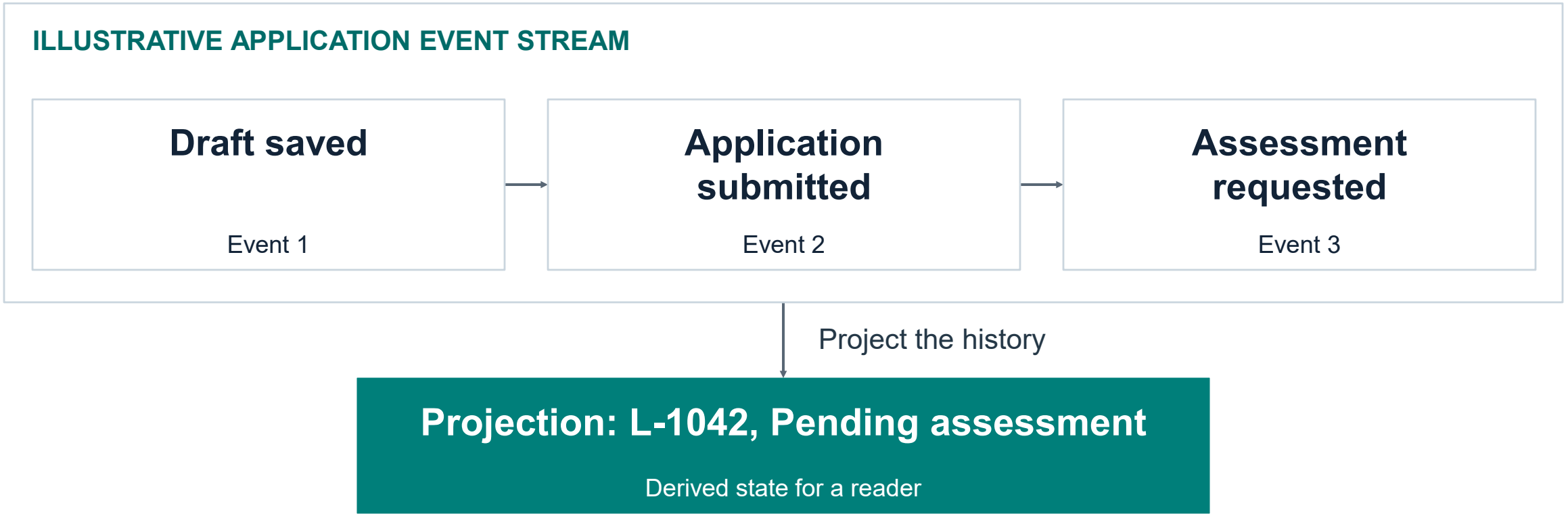
Read side

Show application progress



A shared database is possible. Separate stores add synchronisation and consistency decisions.

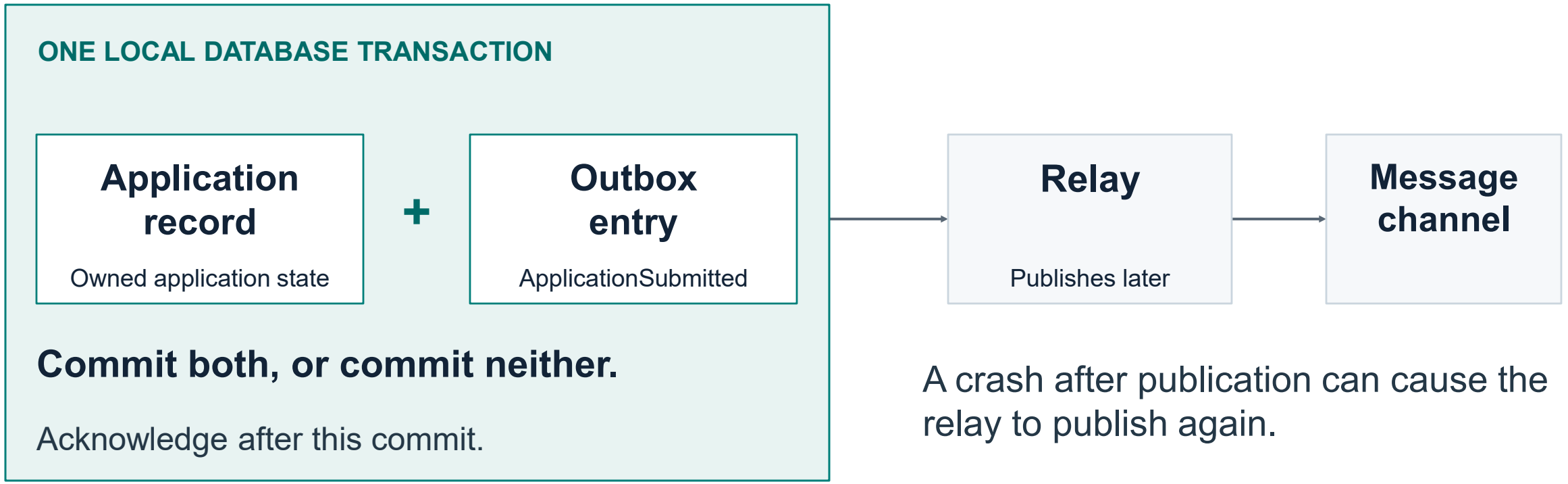
Event sourcing and recorded history



The event stream is authoritative. Published messages or ordinary logs alone do not establish event sourcing.

RELIABLE HANDOFF

Transactional outbox



The outbox closes the save/publish gap. Consumers still need duplicate handling.

RELIABLE HANDOFF

Idempotency and safe retries

Request retry

Reuse the same customer, key and payload.

Return the original application reference.

Define retention and changed-payload rules.

Message redelivery

Use a stable message identifier.

Commit the marker with the local update.

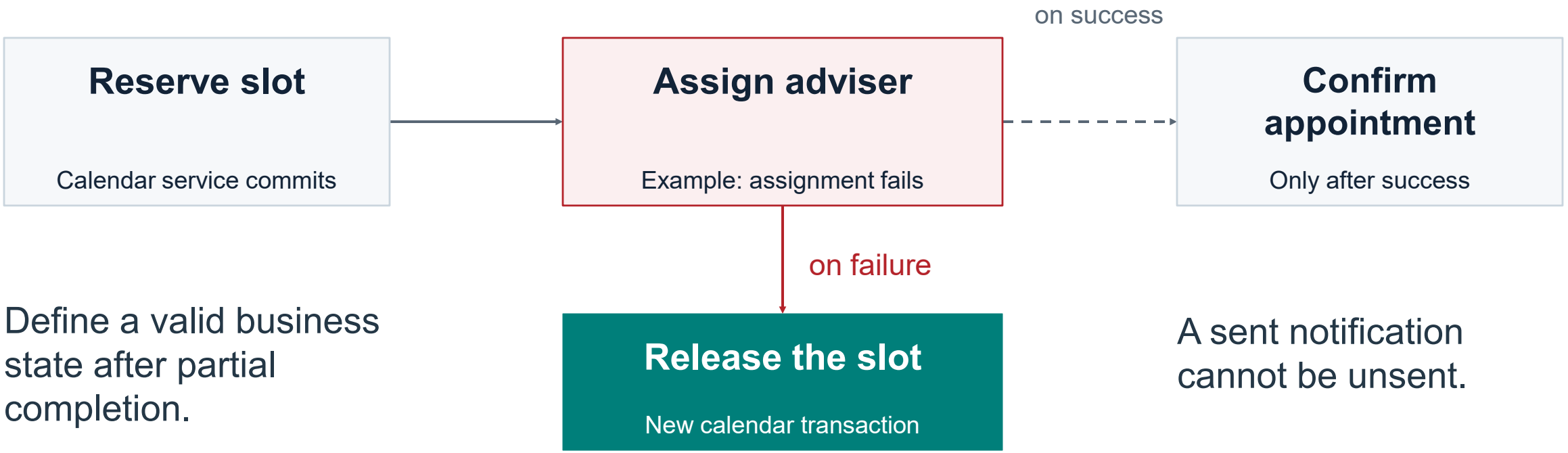
External calls need their own retry contract.

Idempotency protects a defined business effect within an agreed boundary.

SEPARATE BANKING ILLUSTRATION

Saga and business compensation

Example: booking an appointment across services



Define a valid business state after partial completion.

A sent notification cannot be unsent.

Compensation is an explicit business action and can fail. It cannot undo every effect.

Anti-corruption layer

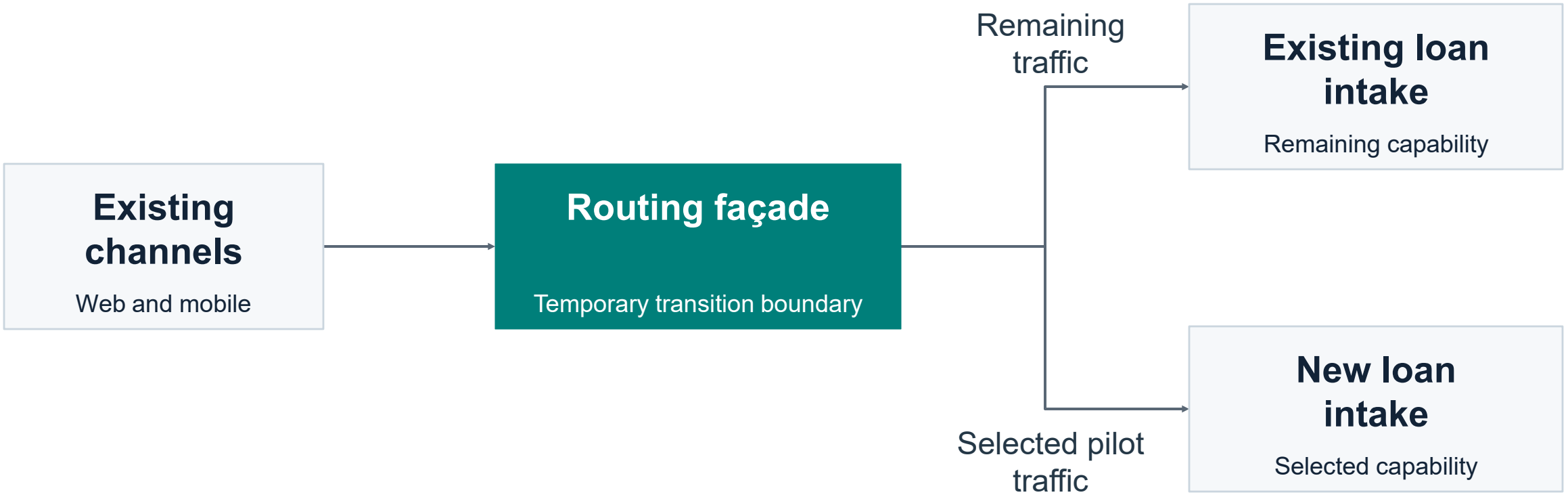


Unknown outcome codes need explicit handling. They must not become invented approvals.

Use when models have materially different meanings. Translation needs contract and version tests.

FUTURE EXTENSION OF THE CASE

Strangler Fig migration



Cutover needs data ownership, reconciliation and a feasible rollback plan. The existing credit service remains in place.

Timeouts, retries and circuit breakers

Use bounded waiting and safe recovery when a dependency fails.

A credit decline is a business outcome. Technical unavailability is a different condition.

Timeout

Bound how long an attempt may wait.

Retry

Retry eligible operations within a budget.

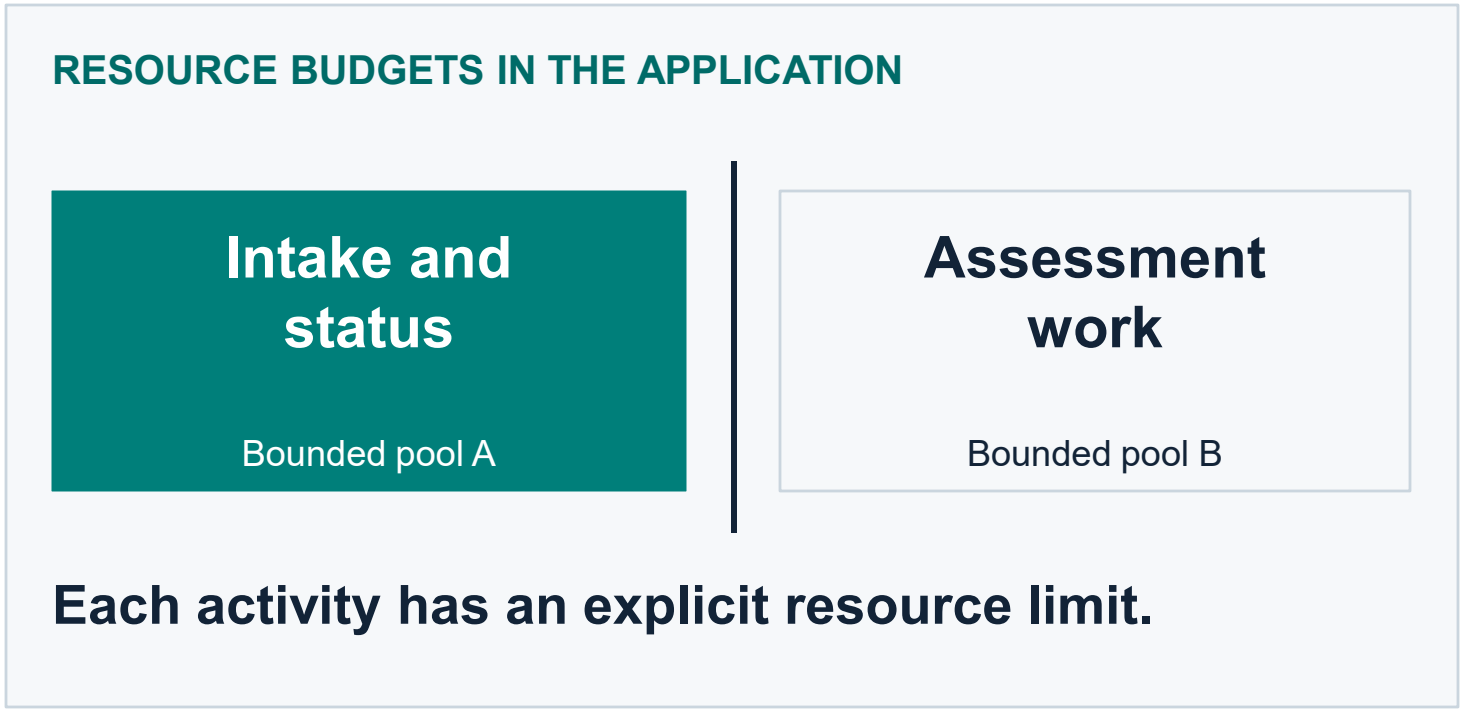
Breaker states

Closed: call. Open: block. Half-open: probe.

Customer outcome

Keep accepted work as “Pending assessment”.

Bulkheads and resource isolation



Protection

Slow assessment should not consume resources reserved for intake.

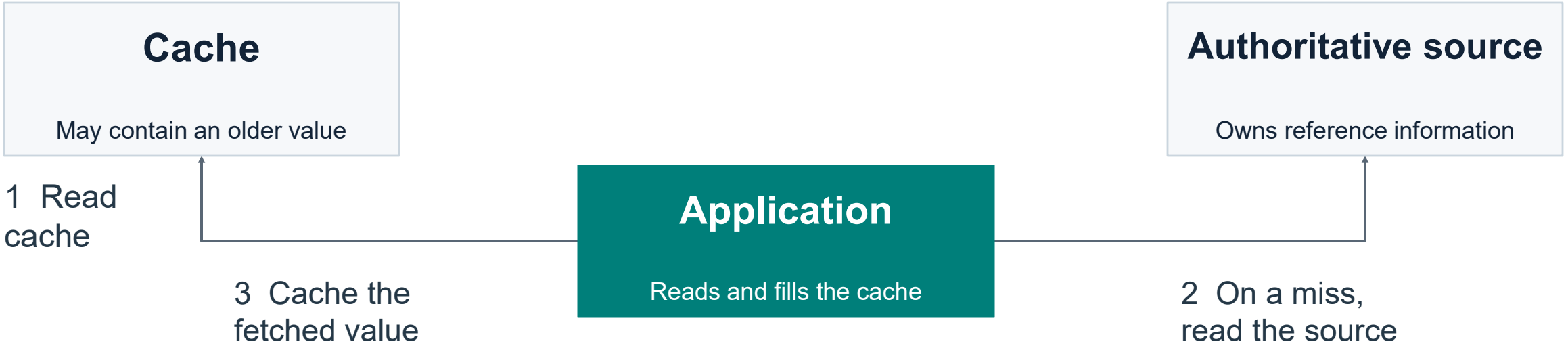
Remaining coupling

Database pools, memory and downstream quotas may still be shared.

Validation: saturate assessment work and measure whether the intake acknowledgement target still holds.

Cache-aside and data freshness

Example: product reference information



Freshness policy: allowed age, expiry, invalidation and cache failure.

Cache-aside can reduce repeated source reads. It does not guarantee fresh data.

A pattern combination for the pilot

Need	Candidate pattern	Resulting responsibility
One pilot delivery team	Modular monolith	Explicit modules with a common release
Existing credit interface	Ports and adapters	Application port mapped to the credit API
Reliable handoff	Outbox and idempotency	Atomic local save; safe handling of repeats
Slow dependency	Timeout, breaker, bulkhead	Bounded waiting and resource use

This is a proposal to test. Every pattern needs a reason, an owner and evidence.

Anti-patterns and their consequences

Anti-pattern	Consequence	Better response
Pattern shopping	Complexity without a clear benefit	Name the problem and a simpler option
Distributed monolith	Services still require joint releases	Test contract and release independence
Shared writable data	Changes bypass another owner’s rules	Use the owning capability’s interface
Unlimited retries	An outage creates more dependency load	Bound deadlines, retries and backoff

Ask for a reason and observable evidence before accepting added complexity.

DESIGN REVIEW

Best practices with evidence

01 **Business purpose**

State the driver, accountable owner and simpler alternative.

02 **Contracts and data boundaries**

Define schemas, errors, authorisation and data ownership.

03 **Failure and performance evidence**

Test crashes, duplicates, load and recovery against agreed targets.

04 **Operability and change**

Track pending work, record decisions and test compatibility.

GROUP EXERCISE

12 MINUTES

Exercise: a crash during handoff

Scenario: one pilot team

A crash occurs after the application is saved but before its event is published.

After restart, a relay may deliver the same event twice.

Your output

01 Choose patterns and explain their fit.

02 Draw the local transaction boundary.

03 State the customer-visible outcome.

04 Specify two failure tests.

API receipt target: p95 ≤ 2 s at 20 valid submissions/s.

With healthy intake/storage. Acknowledge only after durable commit.

EXERCISE DEBRIEF

Debrief: a justified pattern choice

01 **Local atomicity**

Commit the application and outbox entry in one database transaction.

02 **Safe repetition**

Protect local effects. External calls need their own idempotency contract.

03 **Customer meaning**

Return a reference after commit; show “Pending assessment” while delayed.

04 **Evidence and scope**

Test crash recovery and redelivery. Justify the deployment structure.

REFERENCE MATERIAL

References and further reading

Boundaries and application structure

Alistair Cockburn and Eric Evans

- Ports and adapters Domain-Driven Design Reference

Messages and distributed work

Gregor Hohpe, Bobby Woolf and Chris Richardson

- Enterprise Integration Patterns Microservices pattern catalog

Trade-offs and applied guidance

Martin Fowler and Microsoft Architecture Center

- Microservice trade-offs Cloud design patterns

Primary references. Pattern-specific links and teaching context appear in the speaker notes.